a. Identify/Discover a Use Case

-Use Case Name: Solve Puzzle (Process Sudoku Puzzles)

-Primary Actor: Student

-Secondary Actor: Puzzle files

b. Brief Description for the Use Case

The user runs the Sudoku Solver application, which automatically loads the puzzle files, validates the starting numbers, and finds all possible solutions using a recursive backtracking algorithm.

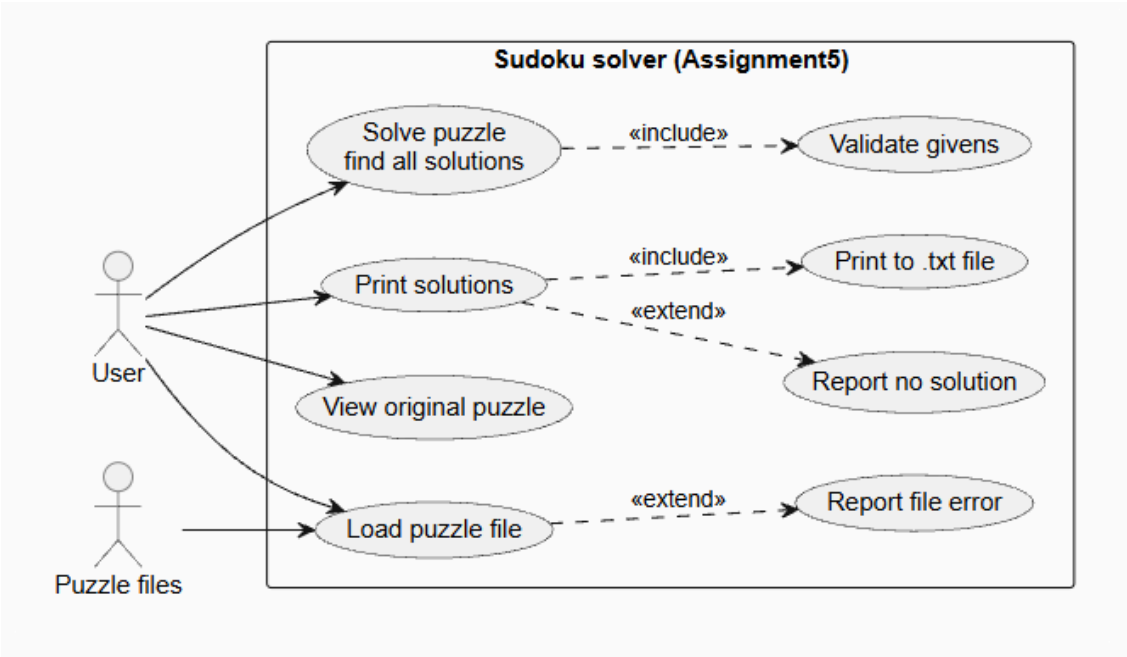
c. Step-by-Step Flow (Outline) of Events

1. The user executes the program.
2. System loads the puzzle file from the same directory
3. System processes the original puzzle configuration.
4. System uses recursive backtracking to find all solutions for the current puzzle.
5. System prints all solutions found to a SolutionN.txt file (or a "No solution found" message) to the matching output file.
6. System repeats the process for the remaining puzzle files (1 through 5).

d. Detailed Description:

The program loops through each of the five puzzle files and reads the grid data safely. Starting at the first puzzle applies the recursive solver to compute all valid completions, writing the output to the corresponding solutionN.txt files. If no valid solutions are found, instead "No solution found" is put in the file. Then the slate is wiped clean to read the next puzzle, repeating until all solutions of all puzzles are found and put into their respective .txt files.

AI use - for this section, I used Google's Gemini to generate PlantUML (https://plantuml.com/) code for a diagram based on the context above:


Sudoku solver (Assignment5)
Solve puzzle
find all solutions
«include»
Validate givens
Print solutions
«include»
Print to .txt file
«extend»
Report no solution
View original puzzle
Load puzzle file
«extend»
Report file error
User
Puzzle files